

Decision Making and Looping in Python

Indentation in Python refers to the (spaces and tabs) that are used at the beginning of a statement. The statements with the same indentation belong to the same block. Instead of brackets in decision making statements Python is using indentation

Python provides various types of conditional statements:

- if statement
- if... else statement
- elif statement

if statement

In [2]:

```
a = 1

if a > 10:
    print("a is greater")
```

if... else statement

In [6]:

```
a = 2
b = 3

if a > b:
    print("a is greater")
else:
    print("b is greater")
```

b is greater

elif statement

In [7]:

```
a = 2
b = 3

if a > b:
    print("a is greater")
elif a == b:
    print("both are equal")
else:
    print("b is greater")
```

b is greater

Single statement Condition

If the block of an executable statement of if - clause contains only a single line, programmers can write it on the same line as a header statement.

In [8]:

```
a = 15  
  
if (a == 15): print("The value of a is 15")
```

The value of a is 15

Looping in Python

- while loop
- for loop

moreover it comes with loop control statements like break, continue and pass.

In [1]:

```
count =1  
  
while count < 6 :  
    print (count)  
    count+=1
```

1
2
3
4
5

for loop

The for statement in Python differs a bit from what you may be used to in C. Giving the user the ability to define both the iteration step and halting condition (as C), Python's for statement iterates over the items of any sequence (a list or a string), in the order that they appear in the sequence.

The range() Function

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax range(start, stop, step) start Optional. An integer number specifying at which position to start. Default is 0 stop Required. An integer number specifying at which position to stop (not included). step Optional. An integer number specifying the incrementation. Default is 1

In [2]:

```
x = range(3, 6)
for n in x:
    print(n)
```

```
3
4
5
```

In [5]:

```
x = range(5)
for n in x:
    print(n)
```

```
0
1
2
3
4
```

for loop with else

interestingly, Python allows using an optional else statement along with the “for” loop. The code under the else clause executes after the completion of the “for” loop. However, if the loop stops due to a “break” call, then it’ll skip the “else” clause.

In [3]:

```
for i in range(5):
    print(i)
else:
    print("No items left.")
```

```
0
1
2
3
4
No items left.
```

In [7]:

```
for i in range(5):
    if i==3:
        break
    print(i)
else:
    print("No items left.")
```

```
0
1
2
```

break and continue in Python

In Python, break and continue statements can alter the flow of a normal loop. The break statement terminates the loop containing it. The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

In [8]:

```
for i in "name":  
    if i=='m':  
        break  
    print(i)  
print("loop over")
```

n
a
loop over

In [1]:

```
for i in "name":  
    if i=='m':  
        continue  
    print(i)  
print("loop over")
```

n
a
e
loop over

Python pass statement

It is used as a placeholder for future implementation of functions, loops, etc. In Python programming, the pass statement is a null statement. The difference between a comment and a pass statement in Python is that while the interpreter ignores a comment entirely, pass is not ignored. However, nothing happens when the pass is executed. It results in no operation (NOP).

In [6]:

```
sequence = ['p', 'a', 's', 's']  
for val in sequence:  
    pass
```

Try it yourself

1. write a program to print maximum of three numbers.
2. write a program to print sum and average of n numbers. Value of n is entered by user at run time.
3. write a program to check given number is odd or even.

In [1]:

```
a=2
b=3
a,b=b,a
print(a)
print(b)
```

3

2

In [9]:

```
str1="Python Basics"

#Str1[7]='b'
print(len(str1))
```

13

In []: